1. Structure of a JWT

A JWT consists of three Base64Url-encoded parts separated by dots: `Header.Payload.Signature`.

- **Header:** Specifies the token type and the cryptographic algorithm used for signing (e.g., `{"alg": "HS256", "typ": "JWT"}`).
- **Payload (Claims):** Contains the actual data or claims about the user (e.g., `{"sub": "user123", "role": "ADMIN", "exp": 1781234567}`).
  - *Crucial note:* Payloads are only Base64 encoded, **not encrypted**. Anyone can decode them, so never put passwords or sensitive PII inside.
- **Signature:** Created by taking the encoded header, encoded payload, a secret key (or private key), and applying the algorithm. This ensures the token cannot be tampered with.

- **Certificate Authority (CA):** A trusted third-party organization (like Let's Encrypt or Google Trust Services) that rigorously verifies server identities and signs **Digital Certificates**.
- **Digital Certificate:** A verified file containing the server's domain name, its **Public Key**, and a cryptographic signature from the CA. Because modern clients (browsers and operating systems) come pre-loaded with a list of trusted root CA certificates, they can mathematically verify that the server's certificate is authentic.

2. The Cryptographic Foundations: Public and Private Keys

Asymmetric cryptography uses a mathematically linked key pair:

- **Public Key:** Shared openly with anyone. Data encrypted with this key can *only* be decrypted by its matching private key.
- **Private Key:** Kept strictly secret and secure on the server. It is used to decrypt traffic or sign cryptographic proofs.

3. The Trust Anchor: Certificate Authority (CA) and Digital Certificates

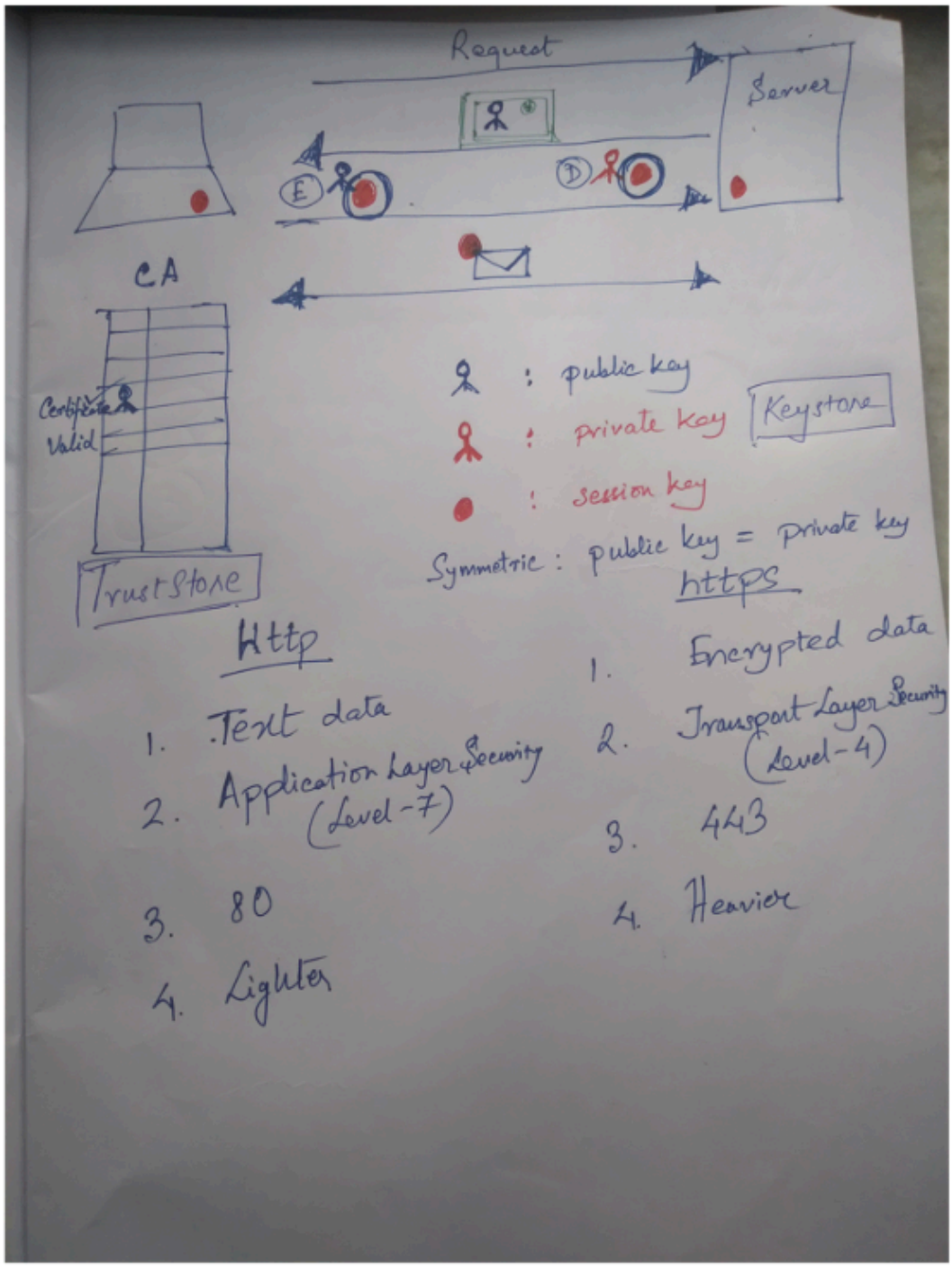
How does a client know that a server is who it claims to be, and not an imposter intercepting traffic?

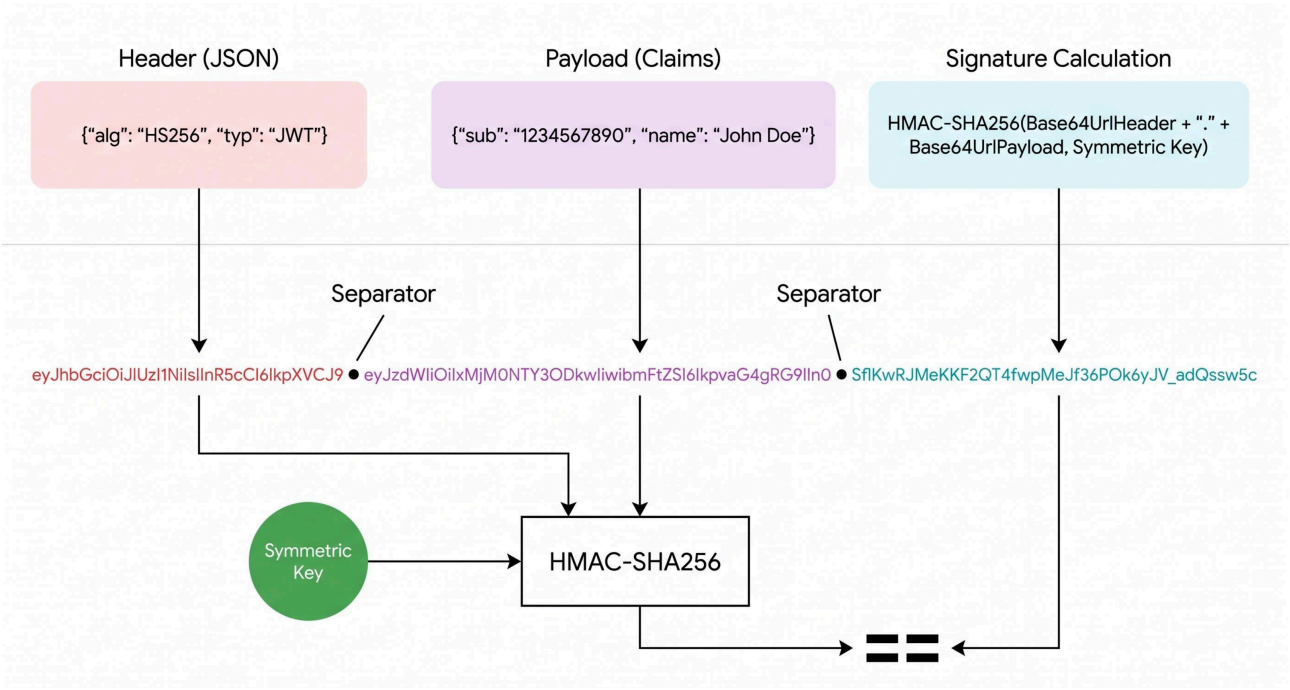
- **Certificate Authority (CA):** A trusted third-party organization (like Let's Encrypt or Google Trust Services) that rigorously verifies server identities and signs **Digital Certificates**.
- **Digital Certificate:** A verified file containing the server's domain name, its **Public Key**, and a cryptographic signature from the CA. Because modern clients (browsers and operating systems) come pre-loaded with a list of trusted root CA certificates, they can mathematically verify that the server's certificate is authentic.

4. Putting It All Together: HTTPS and the TLS Handshake

HTTPS (HTTP Secure) is standard HTTP wrapped inside an encrypted TLS (Transport Layer Security) tunnel. Before any application data or user tokens are exchanged, a **TLS Handshake** occurs:

1. **Hello & Certificate Presentation:** The client initiates contact. The server responds by presenting its **Digital Certificate**.
2. **Certificate Verification:** The client validates the certificate against its trusted CAs. Once confirmed, the client extracts the server's **Public Key**.
3. **Session Key Exchange:** The client generates a random session key, encrypts it using the server's **Public Key**, and sends it back.
4. **Encrypted Tunnel Established:** Only the server, using its secret **Private Key**, can decrypt this message to retrieve the session key. Both parties now share a secure, fast symmetric session key for all subsequent encrypted communication.





How JWT Verification Works over HTTPS ?

When a client makes a request to your Spring Boot REST service, the process flows as follows:

1. The Request Journey: The client attaches the JWT to the HTTP request header over the secure HTTPS connection:
HTTP
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
2. Interception: Spring Security custom filter intercepts the request, strips off the "Bearer " prefix, and isolates the raw token string.
3. Cryptographic Validation: Spring Boot uses its secret key (or private key) to re-calculate the signature of the header and payload. If the calculated signature matches the token's signature, and the expiration (exp) claim is still valid, the token is trusted.
4. Stateless Authorization: Because the token carries its own cryptographic proof of validity, your Spring Boot service does not need to query a database to check user sessions on every single request. It instantly knows who the user is and what roles they hold.